traztech

Cybersecurity & Compliance Consulting

Web Application Penetration Test

Specimen document

Kestrel Data Systems Inc.

Prepared for	Kestrel Data Systems Inc.
Prepared by	Jacob Masse, Principal, Principal, TrazTech Inc.
Date	March 9, 2026
Reference	PT-2026-03

Specimen document

This is a published example of the report TrazTech delivers, not a record of any engagement. The client, the systems, the findings, the evidence and the dates below are invented for illustration. Every host named is under a reserved test domain that cannot resolve. Nothing here describes a real organisation or a real vulnerability in anyone's software.

Scope

Engagement	Web Application Penetration Test
Targets	app.kestreldata.invalid (production web application and its REST API), authenticated and unauthenticated
Testing type	Grey box. Credentialed testing against two role levels, plus unauthenticated testing of the public surface.
Window	2 to 9 March 2026, with retest on 24 March 2026
Tester	Jacob Masse, Principal
Performed	9 March 2026
Reference	PT-2026-03

Anything not named above was out of scope and was not tested. An absence of findings against a system that was never in scope is not a statement about that system.

What was tested

The test areas worked through, and what came out of each. This is here so the report shows its own coverage rather than asking you to infer it from the findings: an area with no findings was still tested, and this is where that is stated.

Area	What was done	Result
Authentication	Credential handling, lockout, multi-factor enforcement across both hosts, password reset lifecycle.	3 findings
Session management	Token issue, rotation, revocation on credential change, cookie flags, idle and absolute expiry.	1 finding

Area	What was done	Result
Authorization	Horizontal and vertical checks across 41 endpoints and 3 role levels, tenant isolation.	1 finding
Input validation	Injection, template handling and output encoding across 63 parameters on 22 forms and endpoints.	1 finding
Business logic	Shipment lifecycle, pricing and quota workflows tested for sequence and state bypass.	No findings
File handling	Upload type and size enforcement, storage path traversal, download authorization.	No findings
API surface	Enumeration, mass assignment, rate limiting and error handling across the documented v1 surface.	3 findings
Transport and headers	TLS configuration and version, certificate chain, security response headers on both hosts.	1 finding
Client side	Third-party script loading, storage of sensitive values, postMessage handling.	1 finding

Method

Testing follows the OWASP Testing Guide and the Penetration Testing Execution Standard. Automated tooling is used to find the known patterns quickly; everything that follows is manual, because business logic and authorisation are where scanners are blind.

- **Reconnaissance.** Passive and active mapping of the target application, its technologies, entry points and attack surface.
- **Mapping.** Crawling, endpoint enumeration, authentication flow mapping and API schema analysis against the documented surface.
- **Vulnerability discovery.** Automated scanning layered with manual testing. Tooling catches the known CVE patterns; manual work targets business logic, chained conditions and access control, which is where scanners are blind.
- **Exploitation.** Controlled exploitation of confirmed conditions, only far enough to establish impact and severity. No destructive testing and no data exfiltration beyond what evidences a finding.
- **Reporting.** Every finding documented with the request that produced it, the log line that recorded it, reproduction steps, a severity rating and specific remediation.
- **Retest.** Validation of remediated findings within a 30-day window after delivery, with the verification evidence recorded against each finding.

No destructive testing was performed. No production data was exfiltrated beyond the minimum needed to evidence a finding, and every record touched is named in the evidence under the finding that required it.

Summary of findings

Critical	High	Medium	Low	Info	Total
1	2	3	2	1	9

9 findings. 7 closed and verified on retest, 2 still open at the date of this report. Severity is assigned from impact in this environment, not from the CVSS base score alone: a medium in one architecture is a critical in another. Both numbers are published against every finding so you can see where the two differ and argue with the one we chose.

Findings at a glance

Ref	Finding	Severity	CVSS	Status
PT-2026-03-001	Broken object-level authorization on shipment records	Critical	9.1	Retest passed
PT-2026-03-002	Administrative console reachable without multi-factor authentication	High	8.1	Retest passed
PT-2026-03-003	Session token remains valid after password change	High	7.4	Retest passed
PT-2026-03-004	Stored cross-site scripting in the shipment reference field	Medium	6.1	Retest passed
PT-2026-03-005	Verbose error responses disclose stack traces and library versions	Medium	5.3	Retest passed
PT-2026-03-006	Password reset tokens do not expire	Medium	5.3	Retest passed
PT-2026-03-008	Rate limiting absent on authentication endpoints	Low	3.7	Not retested
PT-2026-03-007	Security headers absent on the application host	Low	3.1	Retest passed
PT-2026-03-009	Third-party JavaScript loaded without integrity checking	Info	0.0	Not retested

Findings in detail

PT-2026-03-001 · Broken object-level authorization on shipment records

Severity	Critical **CVSS 9.1** CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N
Affected	GET /api/v1/shipments/{id} on app.kestreldata.invalid
Status	Retest passed retested 24 March 2026

What we found. The shipment endpoint checks that the caller is authenticated but never checks that the shipment belongs to the caller's organisation. Any signed-in user of any tenant can read any shipment record by its identifier, including consignee names, addresses and declared values. Identifiers are sequential and the endpoint is not rate limited, so the whole table is enumerable by one low-privileged account in minutes.

REPRODUCTION

1. Sign in as a standard user in tenant A (test account qa-a@kestreldata.invalid) and capture the session cookie.
2. Request a shipment belonging to tenant A and note the numeric identifier returned.
3. Decrement the identifier by one and repeat the request with the same tenant A cookie.
4. Observe that a record belonging to tenant B is returned in full, with HTTP 200.

EVIDENCE

```
Request, sent with a tenant A session against a tenant B record:

  GET /api/v1/shipments/48117 HTTP/1.1
  Host: app.kestreldata.invalid
  Cookie: kds_session=<tenant A session>
  Accept: application/json

Response:

  HTTP/1.1 200 OK
  Content-Type: application/json

  {
    "id": 48117,
    "tenant_id": 91,                <-- caller is tenant 44
    "consignee": "[redacted, tenant B customer]",
    "address": "[redacted]",
    "declared_value": 18400.00,
    "status": "in_transit"
  }

Application log for the same request. The authorization filter never runs:

  10:42:07.118 INFO  auth.session   resolved user=2201 tenant=44
  10:42:07.118 DEBUG route.match    GET /api/v1/shipments/:id
  10:42:07.119 DEBUG repo.shipment  findById(48117)
  10:42:07.121 INFO  http.response  200 6ms
```

```
Compare an endpoint that is correctly scoped, where the filter is present:

  10:44:52.006 INFO  auth.session   resolved user=2201 tenant=44
  10:44:52.006 DEBUG authz.tenant   scope check tenant=44 owner=44 allow
  10:44:52.007 INFO  http.response  200 4ms

Enumeration check, stopped deliberately after 50 sequential identifiers:

  ids 48100-48149 requested with one tenant A session
  200 OK  x 50      (0 rejected)
  distinct tenant_id values observed in responses: 12
```

What to do. Add an ownership check server-side on every record fetch, keyed on the authenticated session rather than on any client-supplied tenant field. Apply it as middleware across the whole resource family rather than endpoint by endpoint, since three sibling endpoints share the same repository call. Replace sequential identifiers with opaque ones as a defence in depth, not as the fix.

On retest. Ownership is now enforced in the API gateway and was verified against three tenants. A cross-tenant identifier returns 404 rather than 403, so the endpoint no longer confirms that a record exists.

RETEST EVIDENCE

```
Same request, same tenant A session, after the fix:

  GET /api/v1/shipments/48117 HTTP/1.1
  Host: app.kestreldata.invalid
  Cookie: kds_session=<tenant A session>

  HTTP/1.1 404 Not Found
  Content-Type: application/json

  { "error": "not_found", "correlation_id": "c8f1a2" }

  10:15:03.442 INFO  auth.session   resolved user=2201 tenant=44
  10:15:03.442 WARN  authz.tenant   scope check tenant=44 owner=91 deny
  10:15:03.443 INFO  http.response  404 3ms

Re-run of the enumeration check across the same 50 identifiers:

  200 OK x 4     (the four records tenant A owns)
  404      x 46
```

PT-2026-03-002 · Administrative console reachable without multi-factor authentication

Severity	High **CVSS 8.1** CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:H
Affected	admin.kestreldata.invalid
Status	Retest passed retested 24 March 2026

What we found. The administrative console accepts a username and password alone. Multi-factor authentication is enforced on the main application but was never extended to the admin host, and four of the eleven accounts holding administrative roles have no second factor enrolled. A single credential from a phishing email or a reused password reaches full tenant administration.

REPRODUCTION

1. Browse to admin.kestreldata.invalid and sign in with a valid administrative credential.
2. Observe that no second factor is requested and the session is issued immediately.
3. Compare with app.kestreldata.invalid, where the same account is challenged for a TOTP code.

EVIDENCE

```
Admin host. Authentication completes in one step:

  POST /session HTTP/1.1
  Host: admin.kestreldata.invalid

  HTTP/1.1 302 Found
  Location: /dashboard
  Set-Cookie: kds_admin=<session>; Secure; HttpOnly; SameSite=Lax

Main application, same account, second factor demanded:

  POST /session HTTP/1.1
  Host: app.kestreldata.invalid

  HTTP/1.1 200 OK
  { "state": "mfa_required", "methods": ["totp"] }

Identity provider enrolment, read from the admin console's own user list:

  role=tenant_admin   accounts=11
    with second factor enrolled ....... 7
    without ........................... 4
    last sign-in over 180 days ago .... 2
```

What to do. Enforce multi-factor on the administrative console for every account without exception, and make enrolment a precondition of holding the role rather than a prompt that can be deferred. Audit the administrative role membership at the same time; two of the eleven accounts had not signed in for over six months and did not need the role.

On retest. Enforced at the identity provider for the admin application. The two dormant accounts were removed rather than enrolled.

RETEST EVIDENCE

```
  POST /session HTTP/1.1
  Host: admin.kestreldata.invalid

  HTTP/1.1 200 OK
  { "state": "mfa_required", "methods": ["totp","webauthn"] }
```

```
role=tenant_admin   accounts=9
  with second factor enrolled ....... 9
  without ........................... 0
```

PT-2026-03-003 · Session token remains valid after password change

Severity	High **CVSS 7.4** CVSS:3.1/AV:N/AC:H/PR:N/UI:R/S:U/C:H/I:H/A:N
Affected	Session handling, app.kestreldata.invalid
Status	Retest passed retested 24 March 2026

What we found. Changing an account password does not invalidate sessions established before the change. A session captured or retained before a reset continues to work until its own expiry, which is thirty days. This defeats the control a user reaches for first after a suspected compromise.

REPRODUCTION

1. Sign in as a test user and retain the session cookie.
2. From a second browser, change that account's password through the account settings page.
3. Replay the original cookie against an authenticated endpoint.
4. Observe HTTP 200 and a valid response.

EVIDENCE

```
Cookie captured before the password change, replayed 40 minutes after it:

  GET /api/v1/profile HTTP/1.1
  Host: app.kestreldata.invalid
  Cookie: kds_session=<pre-change session>

  HTTP/1.1 200 OK
  { "user_id": 2201, "email": "qa-a@kestreldata.invalid" }

Session store at the time of the replay. The old row is untouched:

  session_id  user_id  issued_at            revoked_at
  a91f...c4   2201     2026-03-11 09:02:14  NULL
  7d20...81   2201     2026-03-11 09:41:55  NULL   <- issued by the password change

  password_changed_at for user 2201: 2026-03-11 09:41:55
```

What to do. Invalidate every other session on password change, and on any change to multi-factor enrolment. Give users a visible list of active sessions with a revoke control, so the recovery path does not depend on contacting support.

On retest. Sessions are revoked on credential change and a session list was added to account settings.

RETEST EVIDENCE

```
GET /api/v1/profile HTTP/1.1
Cookie: kds_session=<pre-change session>

HTTP/1.1 401 Unauthorized
{ "error": "session_revoked" }

session_id  user_id  issued_at            revoked_at
a91f...c4   2201     2026-03-24 10:31:02  2026-03-24 10:33:47
```

PT-2026-03-004 · Stored cross-site scripting in the shipment reference field

Severity	Medium **CVSS 6.1** CVSS:3.1/AV:N/AC:L/PR:L/UI:R/S:C/C:L/I:L/A:N
Affected	Shipment detail view, app.kestreldata.invalid
Status	Retest passed retested 24 March 2026

What we found. The shipment reference accepts and stores markup, which is then rendered without encoding on the shipment detail page. A value saved by one user executes in the browser of any colleague who later opens that shipment. Impact is bounded by the field length and by the same-origin session, but it runs with the viewer's privileges.

REPRODUCTION

1. Create or edit a shipment and set the reference field to a value containing an HTML tag.
2. Save, and note that the value is stored verbatim.
3. Open the shipment detail page as a second user in the same tenant.
4. Observe that the markup is parsed rather than displayed as text.

EVIDENCE

```
Stored value, read back from the API exactly as submitted:

  GET /api/v1/shipments/48104 HTTP/1.1

  HTTP/1.1 200 OK
  { "reference": "<img src=x onerror=\"/* proof marker */\">" }

Rendered page source. The value is interpolated with no encoding:

  <td class="ref"><img src=x onerror="/* proof marker */"></td>

The same value on the shipment list view, which does encode correctly:

  <td class="ref">&lt;img src=x onerror="/* proof marker */"&gt;</td>

One of three render paths was sanitised. No content security policy header was
```

```
present on either page:

  curl -sI https://app.kestreldata.invalid/shipments/48104 | grep -i content-security
  (no output)
```

What to do. Encode on output in the template rather than sanitising on input, since the same field is rendered in three places and only one was handled. Add a content security policy that disallows inline script, which turns this class of defect from exploitable into merely present.

On retest. Output encoding applied at the template layer across all three render paths, and a content security policy deployed in enforcing mode.

RETEST EVIDENCE

```
  <td class="ref">&lt;img src=x onerror="/* proof marker */"&gt;</td>

  content-security-policy: default-src 'self'; script-src 'self'; object-src 'none';
                           base-uri 'self'; frame-ancestors 'none'
```

PT-2026-03-005 · Verbose error responses disclose stack traces and library versions

Severity	Medium **CVSS 5.3** CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
Affected	REST API error handling, app.kestreldata.invalid and staging
Status	Retest passed retested 24 March 2026

What we found. Unhandled exceptions return the framework stack trace, the file path on disk and the versions of several third-party libraries. None of this is directly exploitable. It shortens the work of anyone deciding what to try next, and it is what confirmed the framework version used to reach PT-2026-03-001.

REPRODUCTION

1. Send a request with a type the endpoint does not expect, for example a string where an integer is required.
2. Observe that the 500 response carries the stack trace and dependency versions.

EVIDENCE

```
  GET /api/v1/shipments/not-an-integer HTTP/1.1
  Host: app.kestreldata.invalid

  HTTP/1.1 500 Internal Server Error
  Content-Type: application/json

  {
    "error": "ValueError: invalid literal for int() with base 10: 'not-an-integer'",
    "file": "/srv/kds/app/api/shipments.py", "line": 118,
    "framework": "<web framework> 4.2.3",
    "dependencies": {"orm": "2.0.7", "serializer": "3.14.1"},
```

```
    "trace": ["api/shipments.py:118 in get", "repo/shipment.py:52 in find_by_id"]
  }

The staging host, which is internet-reachable, returned the same shape.
```

What to do. Return a generic error body with a correlation identifier and log the detail server-side. Set the framework to production error handling in every environment reachable from outside, which currently excludes the staging host.

On retest. A generic error envelope was deployed across both hosts, with correlation identifiers retained in the server logs.

RETEST EVIDENCE

```
  GET /api/v1/shipments/not-an-integer HTTP/1.1

  HTTP/1.1 400 Bad Request
  { "error": "invalid_request", "correlation_id": "4b2e77" }

Server side, the detail is retained:

  11:02:41.663 WARN  api.shipments  correlation=4b2e77 ValueError invalid id
```

PT-2026-03-006 · Password reset tokens do not expire

Severity	Medium **CVSS 5.3** `CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:N/A:N`
Affected	`Password reset flow, app.kestreldata.invalid`
Status	Retest passed retested 24 March 2026

What we found. Reset tokens remain valid indefinitely until used. A reset link sitting in an old mailbox, a forwarded message or a mailbox backup remains a working credential months later.

REPRODUCTION

1. Trigger a password reset and capture the emailed token.
2. Wait beyond any plausible validity window.
3. Submit the token against the reset endpoint.
4. Observe that it is accepted.

EVIDENCE

```
Token issued 2026-02-04, submitted 2026-03-09, 33 days later:

  POST /reset/confirm HTTP/1.1
  Host: app.kestreldata.invalid

  token=<issued 2026-02-04>&password=<new>
```

```
  HTTP/1.1 200 OK
  { "state": "password_updated" }

Token store. No expiry column is populated:

  token_id  user_id  created_at           used_at  expires_at
  f42c...   2208     2026-02-04 14:19:02   NULL     NULL

Three further tokens for the same account were issued in the interim and all
remained valid at the same time.
```

What to do. Expire reset tokens after a short window, one hour is conventional, and invalidate any outstanding token when a new one is issued or when the password changes by another route.

On retest. Tokens now expire after sixty minutes and are single use. Issuing a new token invalidates the previous one.

RETEST EVIDENCE

```
  token_id  user_id  created_at           used_at  expires_at
  9a01...   2208     2026-03-24 09:12:40   NULL     2026-03-24 10:12:40

  POST /reset/confirm  (token from 2026-02-04)
  HTTP/1.1 400 Bad Request
  { "error": "token_expired" }
```

PT-2026-03-008 · Rate limiting absent on authentication endpoints

Severity	Low **CVSS 3.7** CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:N/A:N
Affected	POST /api/v1/auth/login
Status	Not retested

What we found. The login endpoint accepts unlimited attempts from a single address. Account lockout exists at ten failures per account, so this does not enable brute force against one account, but it permits credential stuffing across many accounts at speed.

REPRODUCTION

1. Submit repeated failed logins from one address, varying the username each time.
2. Observe that no throttle, delay or challenge is applied.

EVIDENCE

```
200 attempts from one address across 200 distinct usernames, sent over 41 seconds:

  responses: 401 x 200
  429 responses: 0
  mean latency: 88ms  (no back-off applied)
```

```
  challenge presented: never

By contrast, eleven attempts against a single account did lock it:

  attempts 1-10  401 invalid_credentials
  attempt  11    423 account_locked
```

What to do. Rate limit by source address as well as by account, and log the rejections somewhere that alerts. The account lockout already in place is the wrong control on its own, because this attack spreads across accounts rather than concentrating on one.

PT-2026-03-007 · Security headers absent on the application host

Severity	Low **CVSS 3.1** `CVSS:3.1/AV:N/AC:H/PR:N/UI:R/S:U/C:L/I:N/A:N`
Affected	`app.kestreldata.invalid`
Status	Retest passed retested 24 March 2026

What we found. Strict-Transport-Security, X-Content-Type-Options and a frame-ancestors policy are not set. Nothing here is exploitable on its own. Each one removes a precondition an attacker would otherwise have to satisfy.

REPRODUCTION

1. Request any page on the application host and inspect the response headers.

EVIDENCE

```
curl -sI https://app.kestreldata.invalid/

HTTP/1.1 200 OK
server: nginx
content-type: text/html; charset=utf-8
set-cookie: kds_session=...; Secure; HttpOnly; SameSite=Lax

absent: strict-transport-security
absent: x-content-type-options
absent: content-security-policy (frame-ancestors)
```

What to do. Set the three headers at the load balancer so they apply uniformly and cannot be missed by a new service. Add Strict-Transport-Security preloading once the policy has run without incident for a period.

On retest. Headers are set at the load balancer and were confirmed present on both hosts.

RETEST EVIDENCE

```
strict-transport-security: max-age=31536000; includeSubDomains
x-content-type-options: nosniff
content-security-policy: ... frame-ancestors 'none'
```

PT-2026-03-009 · Third-party JavaScript loaded without integrity checking

Severity	Info CVSS 0.0
Affected	`Marketing pages, www.kestreldata.invalid`
Status	Not retested

What we found. Two scripts are loaded from a public content delivery network without a subresource integrity attribute. The marketing pages carry no customer data and share no session with the application, so there is no established impact. Recorded because these pages have previously shared a domain with the application and may again.

REPRODUCTION

1. View the page source and inspect the external script tags.

EVIDENCE

```
<script src="https://<cdn>/analytics/3.2.1/a.min.js"></script>
<script src="https://<cdn>/carousel/1.8.0/c.min.js"></script>

neither tag carries integrity= or crossorigin=
```

What to do. Add integrity attributes, or self-host the two files. Worth doing before any future change that puts these pages back on the application origin.

How severity is rated

Rating	What it means here
Critical	Exploitable now, with direct access to customer data, funds or production control. Fix before the next release.
High	Exploitable with a realistic precondition, or a serious weakness in authentication, authorisation or data handling.
Medium	Requires a chain, a privileged position or user interaction, or materially weakens a control without breaking it.
Low	Limited impact on its own. Worth fixing, and worth fixing sooner if it combines with something else on this list.
Info	No security impact established. Recorded because it informs hardening or because it may change with the architecture.

What this report does not say

A penetration test is a time-boxed assessment of the targets named above, in the state they were in on the dates named above. It is evidence about those systems at that time and nothing more. It is not a certification, it is not an audit opinion, and it does not state that no other vulnerability exists.

Systems change. A finding closed on retest was closed against the fix presented at the time, and a later deployment can reintroduce it. Where a finding is recorded as remediated but not verified, that is stated against the finding and this report should not be read as confirming it.

Questions

Anything in this report can be taken up with TrazTech Inc. at hello@traztech.ca, quoting the reference above. Every finding here is also an item in your portal with its own history, so the register and this document are the same record.